

Task: Real-Time Inference Pipeline

Create an application built on GStreamer that ingests a real-time source, runs inference (specs below) on the stream, and outputs structured metadata.

- Core pipeline logic in C++; Python is welcome for tooling and glue, but the performance-critical path should be C++;
- Built on GStreamer;
- Runs on Linux with a CMake build;
- Ingest a real-time source — file, RTSP, webcam (your choice);
- Run an off-the-shelf facial recognition model — match against a small set of faces you've curated. Model and framework are your choice; no training required.
- Emit structured metadata — detections, bounding boxes, landmarks, identities, whatever fits what you built;
- The pipeline must run continuously without stalling, leaking or degrading under sustained load;
- Handle a changing input (e.g. frame drops, rate changes) gracefully;
- Mention the exact versions of the third party libraries used;

Everything else is your call — we have intentionally not specified the model or framework. Pick what you can justify, and be ready to defend your choices.

Suggestions:

- Limit usage of third party libraries for the core functionality of the task;
- Leverage the computer's hardware as much as possible (CPU cores, GPU);
- Optional stretch: push throughput with frame batching, GPU inference or zero-copy;
- Optional: a tool to parse and visualise the metadata;
- Optional: a benchmark (FPS / latency). Report on where the bottleneck is;
- OS independent code is considered an advantage;
- Strive for code readability and comments;
- Impress us;

We are expecting a solution we can compile on Linux. Preferably a VS Code workspace, CMake or make files. Please deliver a git repository or a zip bundle with a short README explaining how to build it, how to run it, and the decisions you made along the way.

Don't hesitate contacting us about questions and clarifications!